Tâche 1: Implémenter une IA locale en Proxy

Le but de ma première tâche était d'implémenter une alternative à l'application pour pouvoir utiliser une IA générative locale à des buts de développement ou afin de pouvoir l'exploiter sans coût direct.

Étape 1:

Pour commencer, j'ai dû créer un point d'entrée stable, autrement dit, un Proxy.

Le principe de l'architecture est que nous avons React qui reçoit un appel POST vers:

`/api/llm`
`/api/image`

Directement vers le Local Proxy à base de Node.js / Express qui utilisera les IA locales suivantes:

`LM Studio (OpenAI-compatible avec JSON)` → Pour le texte LLM
`ComfyAI (API HTTP, txt2img avec workflow)` → pour la génération d'images IA

`La version de Node.js utilisée est v22.12.0 pour Express`
`La version de npm utilisée est 10.9.0`

J'ai donc créé un dossier /server à la racine du projet, puis effectué les commandes afin de construire le build par défaut de Express et CORS. J'ai également installé nodemon pour me faciliter le développement.

Après avoir créé index.ts dans le dossier server, avoir ajouté des endpoints, set-up express, cors et ajusté tsconfig.json et package.json, mon serveur proxy s'est officiellement lancé.

Étape 2:

J'ai branché le /api/llm vers le LLM local `mistral-7b-instruct` depuis le logiciel LM Studio qui roule depuis un serveur local.

La majorité du travail a été automatisé à l'aide du logiciel LM Studio.

Étape 3:

J'ai initialement installé Automatic1111 pour connecter le endpoint /api/image.

Pour ce faire, il me manquait un `stable diffusion` comme celui initialement prévu n'était plus supporté par ses développeurs. J'ai donc dû aller installer un autre SDXL 0.9

programme pour l'insérer dans le dossier:

`sd_xl_base_0.9.safetensors`

Source: https://huggingface.co/stabilityai/stable-diffusion-xl-base-0.9/tree/main

Il sert à déterminer un modèle de génération d'images.

Problème rencontré:
Quand j'ai lancé Automatic1111, il a essayé de télécharger un vieux "moteur Stable Diffusion" qui n'existe plus sur GitHub.
Du coup, le démarrage cassait et l'API ne pouvait pas se lancer.

Après beaucoup de recherches, j'ai finalement laissé tombé et j'ai opté pour ComfyAI.

Étape 3.1:
Installation de ComfyAI pour la génération IA d'images.

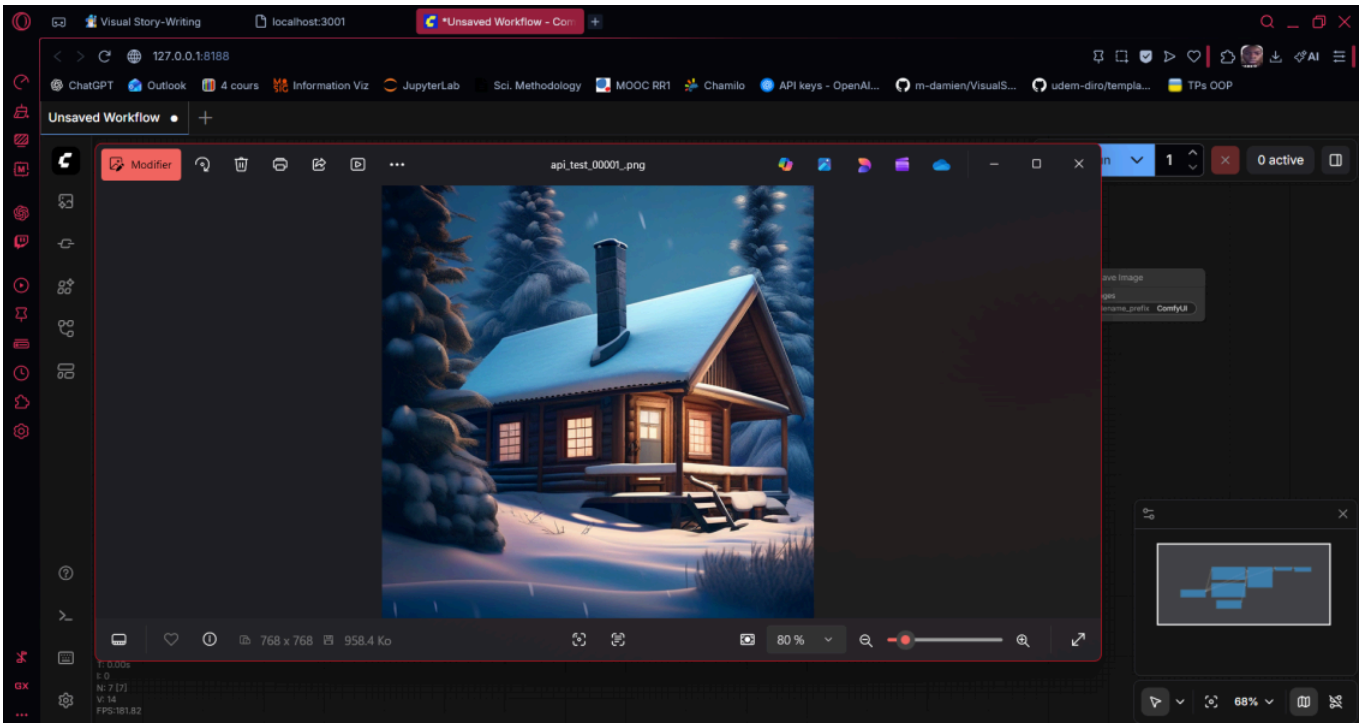
J'ai ajouté `sd_xl_base_0.9.safetensors` dans le dossier
`ComfyAI/ComfyUI/models/checkpoints/

Après avoir installé le API de ComfyAI, et ayant executé ce code pour le faire rouler:
```
## Changez le absolute path à celui de votre machine vers le dossier ComfyAI...

cd C:\Users\andre\OneDrive\Desktop\ComfyAI\ComfyUI..\venv\Scripts\activate
python main.py
```

J'ai pu set-up le API endpoint sur mon serveur Express et avec un fichier de payload.json ajouté au sein de la codebase et la commande CMD suivante:

```
curl -X POST http://127.0.0.1:3001/api/image ^
-H "Content-Type: application/json" ^
--data-binary "@C:\Users\andre\OneDrive\Desktop\VisualStoryWriting-Andrei\payload.json"
```



J'ai pu officiellement générer des images depuis une IA locale avec ComfyAI et mon GPU.

Le code du endpoint est comme d'habitude dans /server/index.ts

Tâche 2: Faire afficher une image générée avec IA sur le frontend depuis un prompt quelconque

Afin de pouvoir éventuellement faire fonctionner le programme qui prend une histoire de l'utilisateur et isole ses éléments pour générer des images de ses personnages, objets et paysages, je dois passer par l'étape élémentaire de simplement pouvoir générer une image IA depuis un prompt quelconque sur le frontend de l'application.

Cette étape fut plutôt directe étant donné qu'elle n'a qu'impliqué beaucoup de boilerplate code qui permet de générer les images depuis un prompt traduit en objet JSON.

Tâche 3: Set-up le LLM de Mistral qui récupère le texte pour générer des entités avec leurs traits et leurs images générées.

Voici les étapes à faire déjà pour débuter, celles-ci ne nécessite que du set-up sans trop de code comme le endpoint a été créé plus haut dans le server Express du fichier index.js

Dans Discover sur LM Studio, installez idéalement `mistral-7b-instruct-v0.2`

Puis allez dans l'onglet où ça vous propose de charger un modèle du local server, et choisissez votre LLM téléchargé à l'instant. Puis faites le petit toggle button pour rendre le status du serveur comme ''Running''.

Dans LM Studio Developer, le serveur devrait être sûr http://127.0.0.1:1234.
Dans la codebase, sur Powershell:

```
npm --prefix server run start
```

puis:

```
$body = @{ model = "mistral-7b-instruct-v0.2" messages = @(@{ role = "user"; content = "Reply with exactly: PROXY OK" }) temperature = 0 } | ConvertTo-Json -Depth 6 Invoke-RestMethod -Uri "http://127.0.0.1:3001/api/llm" -Method Post -ContentType "application/json" -Body $body
```

Si ça ne retourne pas d'erreur, alors tout fonctionne.

Nous ajoutons alors ensuite dans Model.tsx, le aiProvider variable qui permet la distinction entre la key OpenAI et la IA Local.

Suivant ce que j'ai compris du code hérité par défaut, j'ai déduis une méthode pour faire fonctionner notre IA locale que nous allons appeler ''Mistral'' simplement dorénavant afin que celui-ci puisse être entrainé pour générer des images des éléments du texte.

Ce prompt va tester un texte avec beaucoup de description variable sur ses entités:

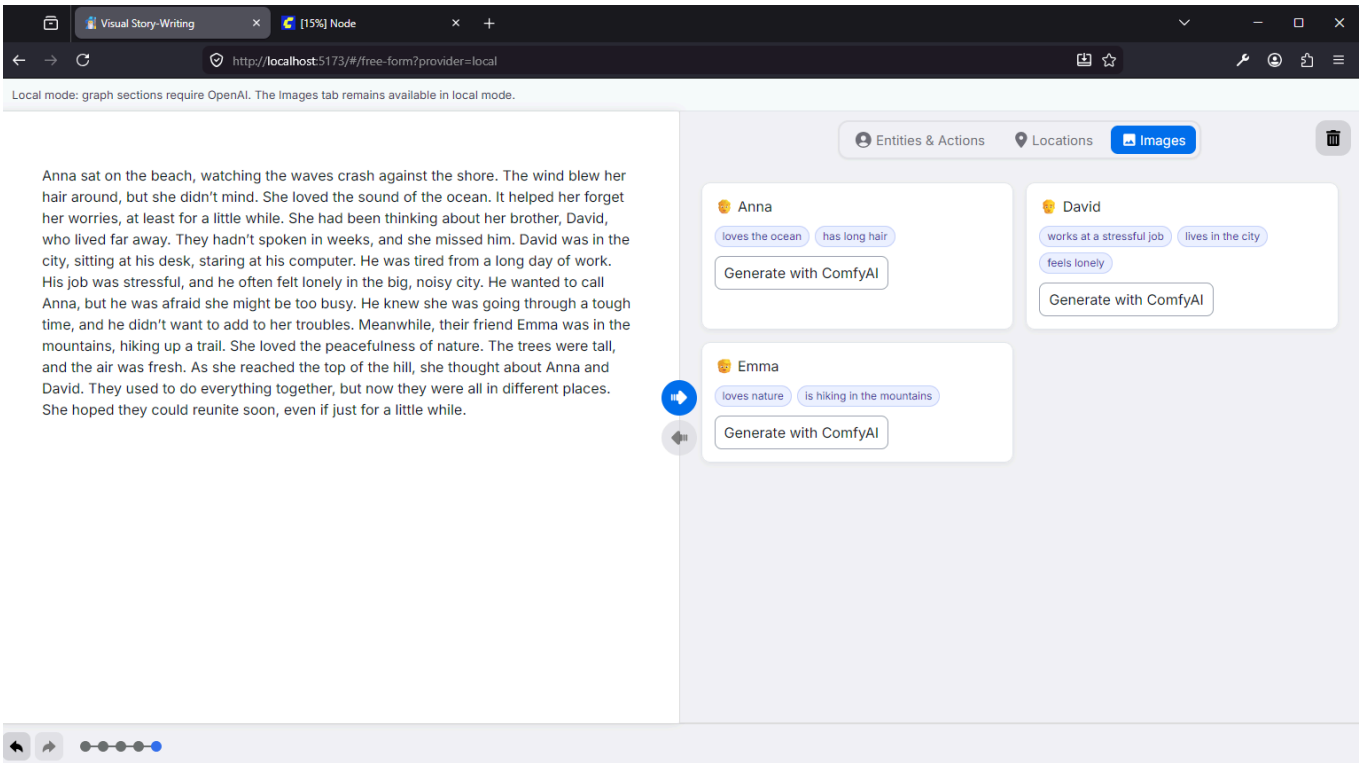
```
Bob, a tall bald man, and grease-stained hands from working as a mechanic, stood on a quiet road beside his rusty red pickup truck. He wore a faded blue jacket and heavy work boots, carrying a worn leather tool bag over his shoulder. Nearby stood Maria, a woman in her early thirties with long dark hair, tired brown eyes, and a thin scar above her eyebrow. She wore a dark green hiking jacket and mud-covered boots. In the shadows at the edge of the forest lay a large gray wolf with thick silver fur and bright amber eyes, its hind leg trapped beneath fallen branches.
```

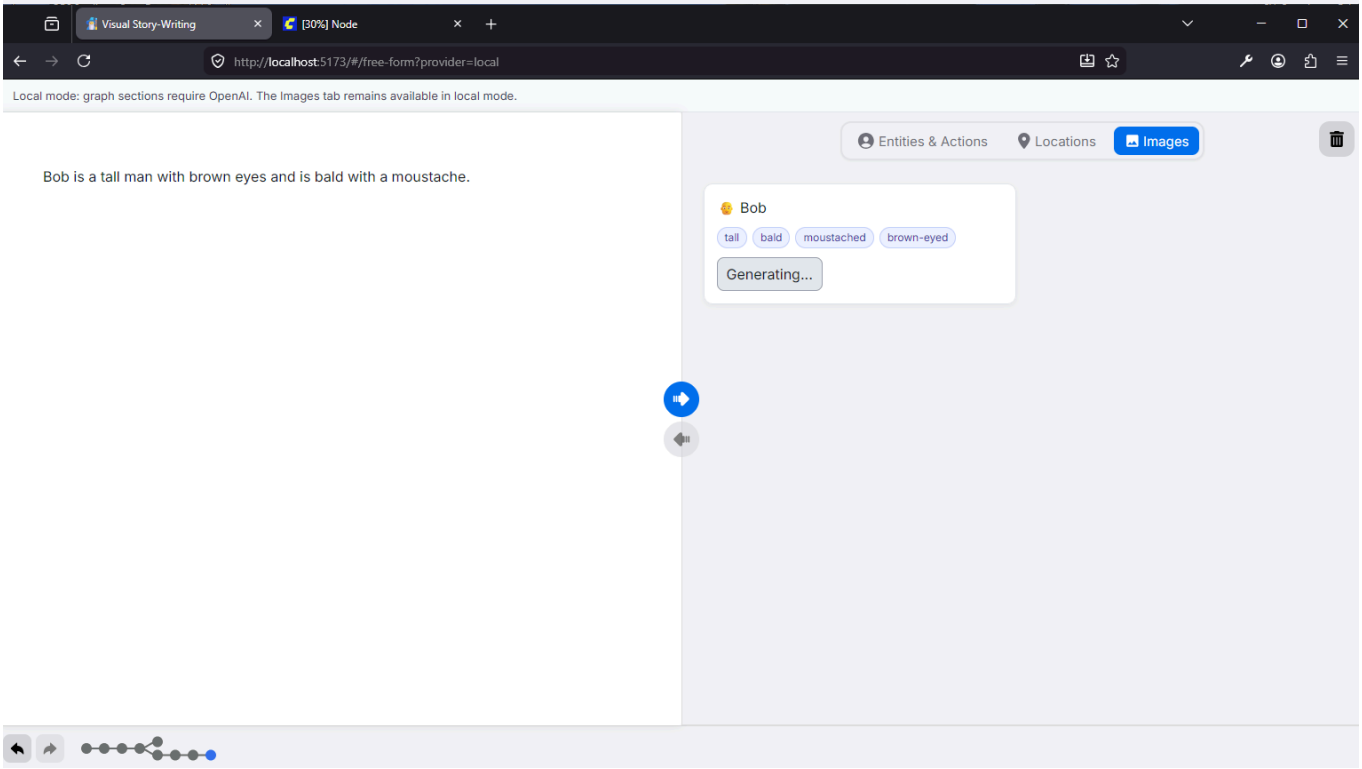
Ce prompt va tester un texte avec peu de description sur ses entités:

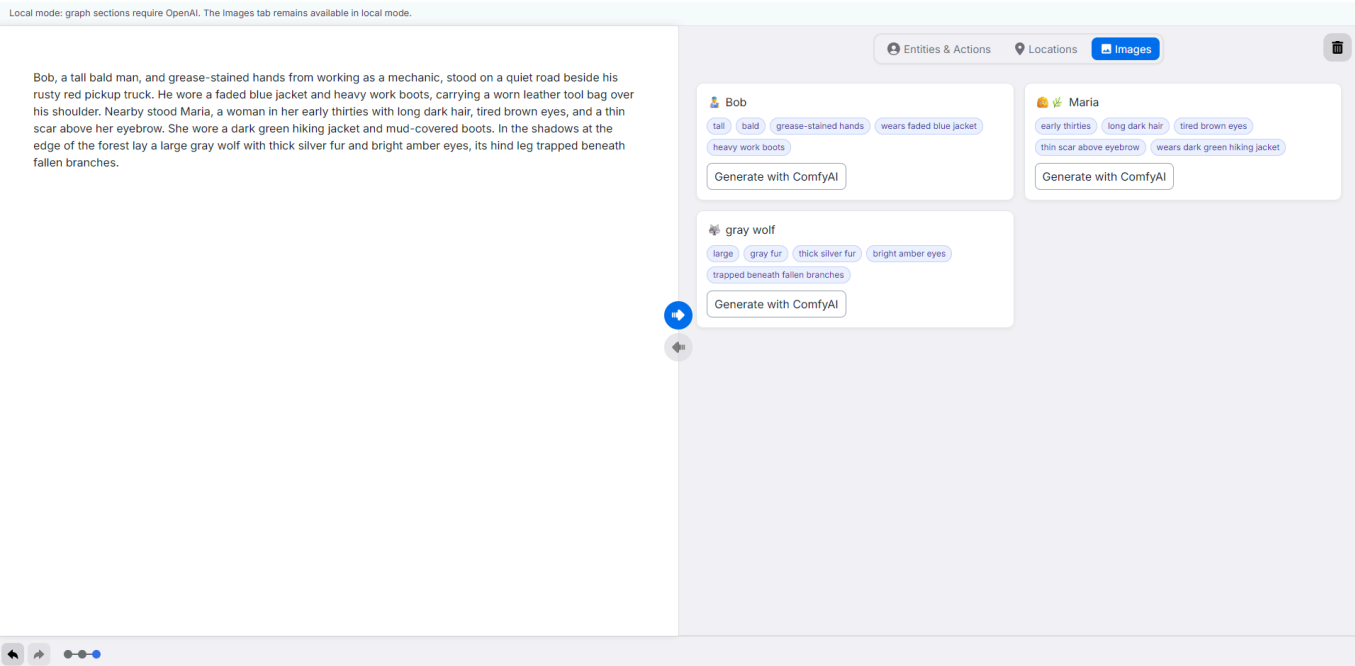
```
Anna, a white woman with long black hair and slanted eyes is sat on the beach, watching the waves crash against the shore. The wind blew her hair around, but she didn’t mind. She loved the sound of the ocean. It helped her forget her worries, at least for a little while. She had been thinking about her brother, David, who lived far away. They hadn’t spoken in weeks, and she missed him. David was in the city, sitting at his desk, staring at his computer. He was tired from a long day of work. His job was stressful, and he often felt lonely in the big, noisy city. He wanted to call Anna, but he was afraid she might be too busy. He knew she was
```

```
going through a tough time, and he didn’t want to add to her troubles. Meanwhile, their friend Emma was in the mountains, hiking up a trail. She loved the peacefulness of nature. The trees were tall, and the air was fresh. As she reached the top of the hill, she thought about Anna and David. They used to do everything together, but now they were all in different places. She hoped they could reunite soon, even if just for a little while.
```

Après des heures perdues sans vains, je suis arrivé à la conclusion que les IA locales manquent terriblement de puissance afin de pouvoir produire le workload que l'application demande.

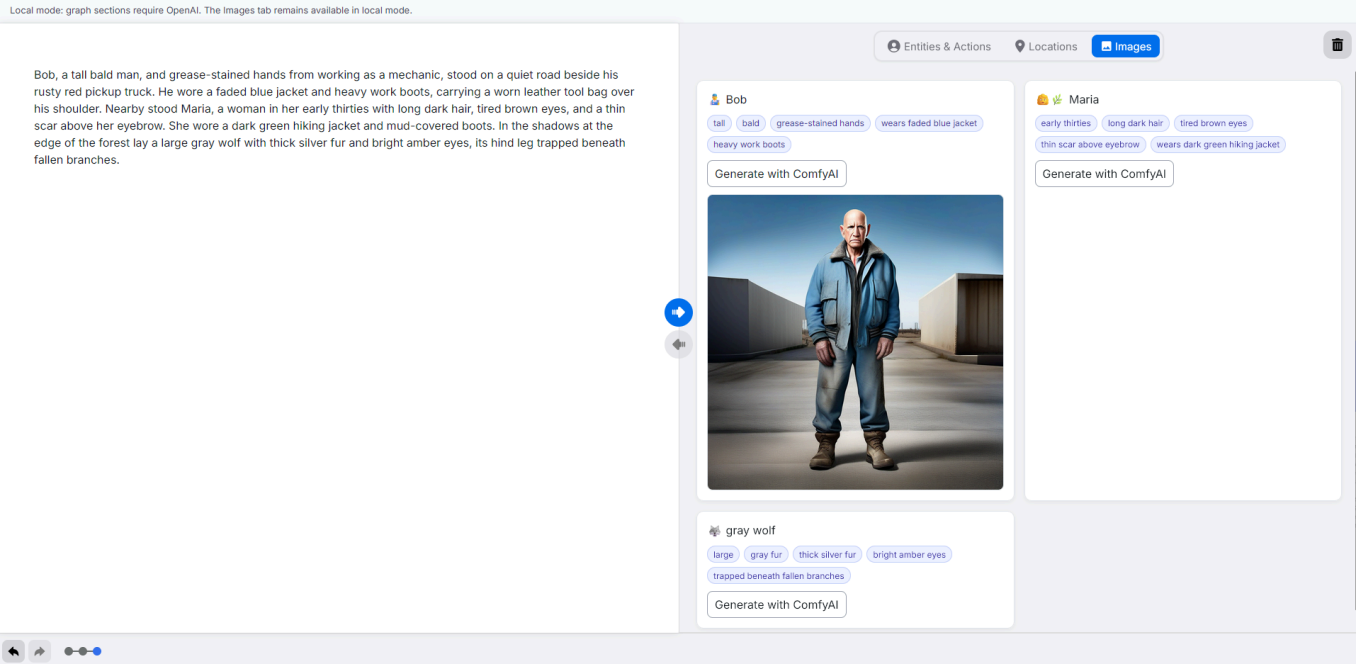

Visual Story-Writing
[15%] Node
http://localhost:5173/#/free-form?provider=local
Local mode: graph sections require OpenAI. The Images tab remains available in local mode.
Anna sat on the beach, watching the waves crash against the shore. The wind blew her hair around, but she didn't mind. She loved the sound of the ocean. It helped her forget her worries, at least for a little while. She had been thinking about her brother, David, who lived far away. They hadn't spoken in weeks, and she missed him. David was in the city, sitting at his desk, staring at his computer. He was tired from a long day of work. His job was stressful, and he often felt lonely in the big, noisy city. He wanted to call Anna, but he was afraid she might be too busy. He knew she was going through a tough time, and he didn't want to add to her troubles. Meanwhile, their friend Emma was in the mountains, hiking up a trail. She loved the peacefulness of nature. The trees were tall, and the air was fresh. As she reached the top of the hill, she thought about Anna and David. They used to do everything together, but now they were all in different places. She hoped they could reunite soon, even if just for a little while.
Entities & Actions
Locations
Images
Anna
loves the ocean
has long hair
Generate with ComfyAI
David
works at a stressful job
lives in the city
feels lonely
Generate with ComfyAI
Emma
loves nature
is hiking in the mountains
Generate with ComfyAI


Visual Story-Writing
[30%] Node
http://localhost:5173/#/free-form?provider=local
Local mode: graph sections require OpenAI. The Images tab remains available in local mode.
Bob is a tall man with brown eyes and is bald with a moustache.
Entities & Actions
Locations
Images
Bob
tall
bald
moustached
brown-eyed
Generating...



Avec la meilleure IA trouvée sur LM Studio, **mistral-7b-instruct-v0.2**

J'ai pu faire fonctionner la section des fonctionnalités que je compte rajouter, cependant, par manque de puissance, ce ne sera que la seule section accessible en mode IA locale. Le code devrait être transitif avec une clé vers l'API d'Open AI, mais ça reste à confirmer une fois que je vais avoir accès à une clé.



En réutilisant le code qui permet à l'IA de modifier le texte à partir de son contenu à droite, et en rajoutant la possibilité d'ajouter et de retirer des traits physiques, j'ai pu implémenter une V1 de ce qui nous permet de modifier l'apparence de nos entités et d'ensuite pouvoir les générer visuellement.

Cela dit, avec une IA locale, tout prompt devant un temps exponentiellement long, mais au moins il fonctionne, cependant on se limitera à fortement conseiller d'utiliser l'API d'OpenAI sinon, les changements à la main demeurent plus rapide que d'attendre que l'IA génère des propositions.

Je tiens également à noter, que dans la boite noir de l'IA locale, elle semble manquer de rigueur lorsqu'on lui demande de façon efficace d'apporter des modifications selon les changements de traits, ça reste à vérifier si ChatGPT sera en mesure d'être plus concise sur la tâche donnée. Cela va de soit pour ComfyAI qui n'est pas parfaite sur tous les petits détails que j'ose espérer ChatGPT saura porter attention, lors de la génération d'images.

Dans `ImageEntitiesExtractor.ts`

```
export class ImageEntitiesExtractor {
  Windsurf: Refactor | Explain | Generate JSDoc | X
  static getPrompt(text: string): string {
    return `${text}

Extract ALL distinct visually representable entities from this story.

For each entity, return:
- name
- emoji
- traits (short visual traits, e.g. old, tall, armored, red-haired)

Rules:
- Return only distinct entities like characters, objects and locations (merge duplicate mentions).
- Exclude actions, emotions, intentions, personality, relationships.
- Return ONLY valid JSON following the required schema.
`;
  }
```

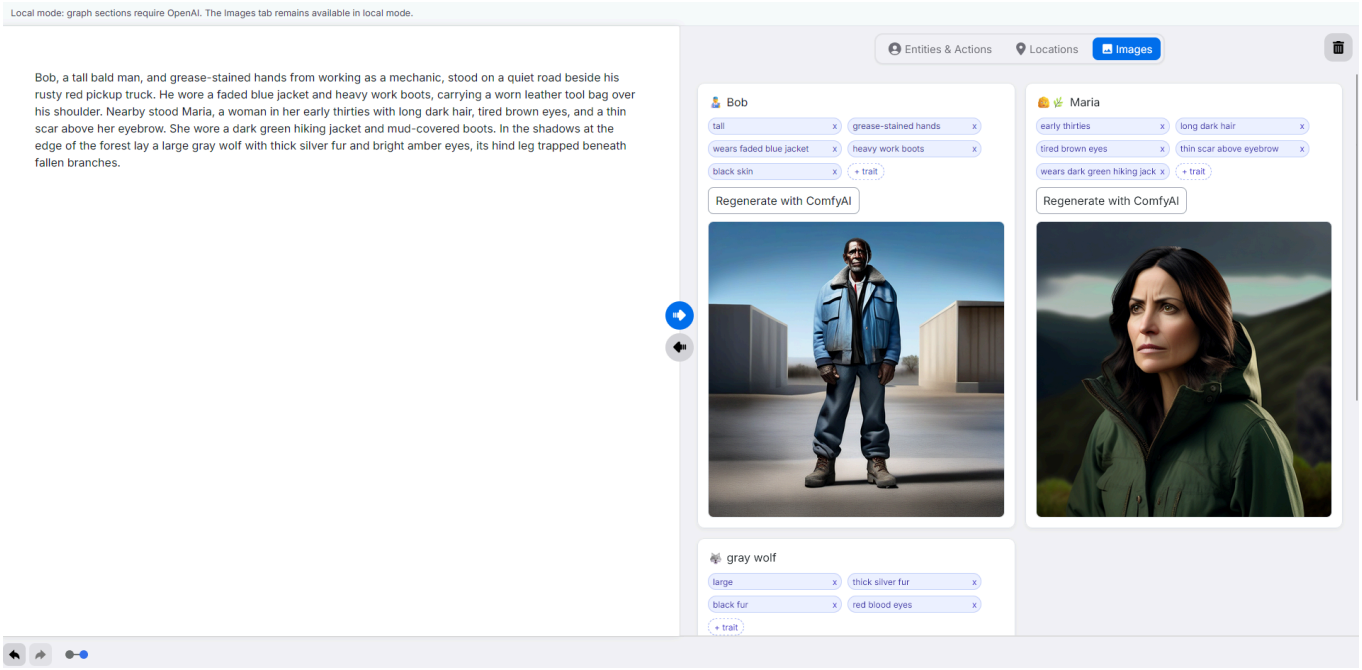
Ce prompt au début de la classe est ce que le LLM reçoit lorsqu'il doit générer les cartes avec les entitiés et leurs traits physiques. Libre de le modifier au besoin à l'avenir, normalement toute la classe est compatible avec du prompt d'OpenAI.

Puis dans `JSONPrompt.tsx` et `ImagesEditor.tsx` nous avons plusieurs variables qui assurent le retour valide des JSON pour la génération des entités et de leur image IA.

Dans la classe `RewriteFromImageTraits`

```
    return (
        `STORY:\n${text}\n\n` +
        `Update only physical descriptions in STORY so each entity matches these traits:\n${entitiesDescription}\n\n` +
        `Rules:\n` +
        `- Make MINIMAL edits only.\n` +
        `- Keep plot, events, order, and locations unchanged.\n` +
        `- Keep names exactly as written.\n` +
        `- Keep the same language and overall style.\n` +
        `- Keep the same number of paragraphs as STORY.\n` +
        `- Keep sentence order and sentence count unchanged.\n` +
        `- Do not add any new events, scenes, or dialogue.\n` +
        `- Only modify short descriptive spans tied to physical appearance.\n` +
        `- Do not output explanations, thoughts, prompt text, or analysis.\n\n` +
        `Output format (strict):\n` +
        `<rewritten_story>\n` +
        `...rewritten full story only...\n` +
        `</rewritten_story>`
    );
}
```

Ceci est le prompt utilisé lorsque je demande à l'IA de modifier le texte et proposer les changements en lien avec les traits physiques ajoutés ou retirés sur un ou plusieurs personnages.



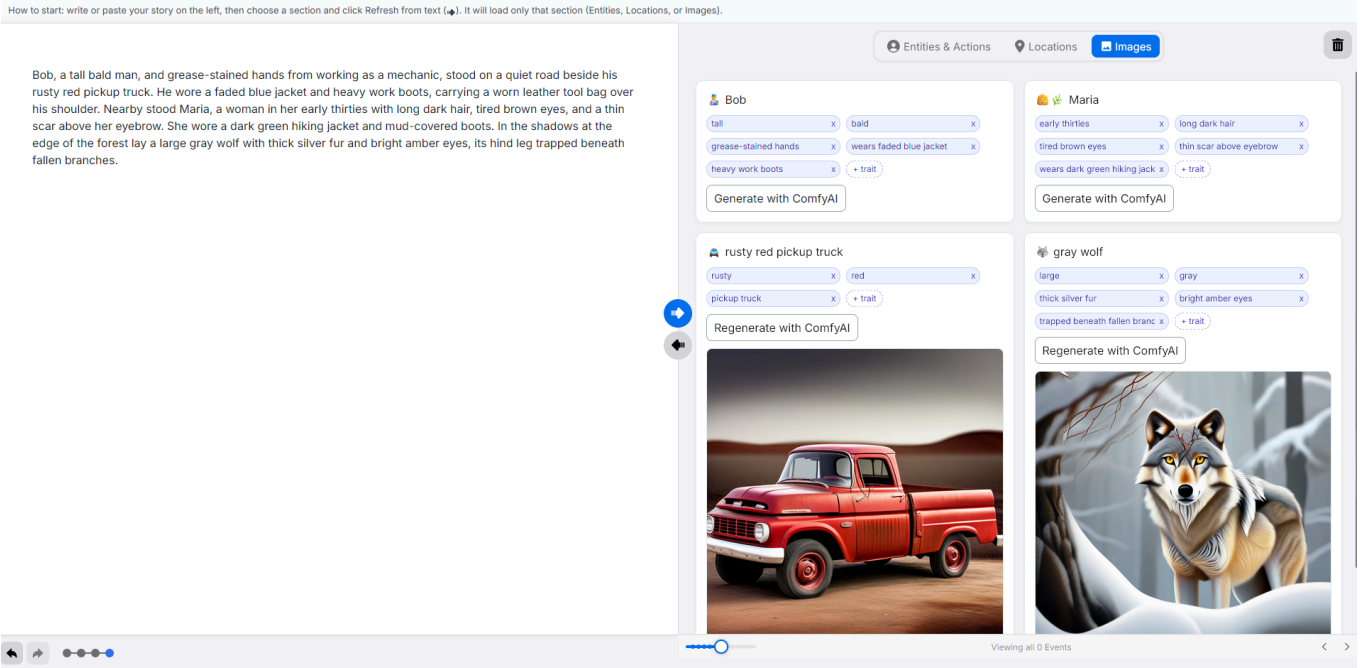
Étonnement, les images se génèrent très rapidement, et lorsque je regénère avec des traits modifiés, c'est également très rapide et bien concis aussi, le loup qui avait les yeux amber, a fini par avoir les yeux rouge en changeant les traits et Bob est passé de blanc chauve à noir avec des cheveux (j'ai retiré le détail d'être chauve).

Il reste qu'à voir si OpenAI sera plus efficace que Mistral pour modifier le texte à partir des nouveaux changements des entités et leurs traits physiques.

Après quelques manip, j'ai tenté de réintégrer toutes les fonctionnalités avec Mistral, en demandant individuellement de les générer.

J'ai finalement rajouté la possibilité de queue les générations d'image si l'utilisateur en demande plusieurs à la volée. J'ai aussi fix un petit détail qui faisait que la voiture n'était

pas considéré comme une entité dans mon prompt. P.S. Toutes les images de code sur le document sont les versions finales durant la tâche et donc avec déjà les bugs corrigés.



Je suis aussi revenu sur mon choix de bloquer les deux autres sections aux IA locales, bien que celles-ci ne fonctionnent pas comme attendu, à voir si on garde.

J'ai finalement ajouté la possibilité de tout supprimer dans la section des Images avec le bouton déjà partiellement implémenté.

Tâche 4: Implémenter l'éditeur de dessin sur image (canvas + masque)

Le but de cette tâche était de permettre à l'utilisateur de dessiner directement sur l'image d'un personnage, puis d'utiliser ce dessin comme guide pour modifier l'image via IA.

Étape 1:

J'ai créé un éditeur de dessin en React basé sur deux canvas superposés, un canvas base (image actuelle du personnage) et un canvas draw (calque de dessin par défaut).

J'ai également ajouté les outils suivants:

- pinceau
- gomme
- sélection de couleur
- taille du pinceau

- undo/redo
- bouton pour effacer entièrement le dessin

Le composant principal utilisé est:
`src/view/imageView/ImagePaintEditor.tsx`

Étape 2:

J’ai implémenté la génération du payload d’édition à partir du canvas. À partir de l’état du dessin, je génère le baseImageDataUrl (image originale), drawLayerImageDataUrl (dessin utilisateur), le maskImageDataUrl (zone à modifier) et editPromptText, qui est texte obligatoire de modification, car la performance des IAs génératives d'images n'est pas rendu suffisante pour comprendre seulement à partir d'un dessin.

Le masque est généré automatiquement à partir des pixels dessinés, avec une légère dilatation autour des traits pour aider l’IA à mieux comprendre la zone d’édition.

Étape 3:

J’ai intégré cet éditeur directement dans la vue des images:

`src/view/imageView/ImagesEditor.tsx`

Chaque carte personnage possède désormais un bouton Edit permettant d’ouvrir l’éditeur. Cependant, au début, la preview fusionnée (image + dessin) pouvait être confondue avec l’image finale. Retournant soit l'image initiale, l'image avec le dessin, ou une génération vraiment divergente de ce qui était attendu.

J’ai séparé clairement les états pour garantir que le dessin reste un calque temporaire et l’image finale affichée ne dépend que du résultat IA.

Tâche 5: Corriger le modèle d’édition (bug critique)

Le but de cette tâche était de corriger un bug critique où l’image avec dessin ne générait pas ce qui était attendu à travers la boîte noir qu'est l'IA OpenAI et ComfyAI.

Étape 1:

J’ai isolé les états de manière stricte l'image de base, le calque de dessin, le masque preview locale fusionnée et image finale IA.

Étape 2:

J'ai ajouté de façon strict de ne pas modifier l'image affichée pendant la regénération, si le backend réussi, que l'image soit remplacé par celle retournée et en cas d'erreur que rien ne change, ce qui arrivait souvent avec le format imgtoimg des IAs. Au moins, l'IA ne retournait jamais le dessin du calque.

Tâche 6: Tentative OpenAI puis migration complète vers ComfyAI

Le but initial était d'utiliser OpenAI pour l'édition d'image, mais cela s'est révélé non fiable dans ce contexte. En effet, après multiples erreurs avec la clé de l'API, le port ou encore les confusions de l'IA, j'ai commencé à chercher la raison de mes problèmes qui ne retournaient pas l'image regénérée attendue.

https://platform.openai.com/docs/api-reference/images/overviewhttps://help.openai.com/en/articles/11128753

**En m'étant informé à travers ces trois sources globalement, j'ai tiré la conclusion que l'API OpenAI ne fonctionne pas pour la transition génération imgtoimg à cause que le modèle utilisé est dall-e-3 qui ne supporte pas les éditions.

J'ai ensuite essayé d'utiliser la variante dall-e-2, mais celui-ci est hyper strict avec les contraintes qui étaient engendrés par les erreurs d'états dans le frontend et le backend.**

La documentation d'OpenAI indique explicitement des limitations en termes de cohérence et de composition pour les modèles d'images, en particulier lors d'éditions précises.

D'ailleurs, avec la fermeture de Sora 2, OpenAI a aussi annoncé le que Dall-E 2 et 3 seront deprecated et que leur support va cesser le 12 mai 2026, ce qui rend impossible une future mise-à-jour améliorant le modèle.

Voici ma source ici:

https://developers.openai.com/api/docs/guides/image-generation

---

Après environ une semaine de tentatives et recherche, j'ai donc décidé d'abandonner OpenAI pour cette feature et de migrer entièrement vers ComfyAI.

Nouvelle route backend:
`POST /api/image/edit`
dans `server/index.ts`

On upload vers Comfy, qui exécute le workflow, effectue du polling avec le endpoint `/history`, la récupération avec `/view` et finalement retourne l'image en base64 au lieu de URL pour augmenter l'efficacité de mes besoins.

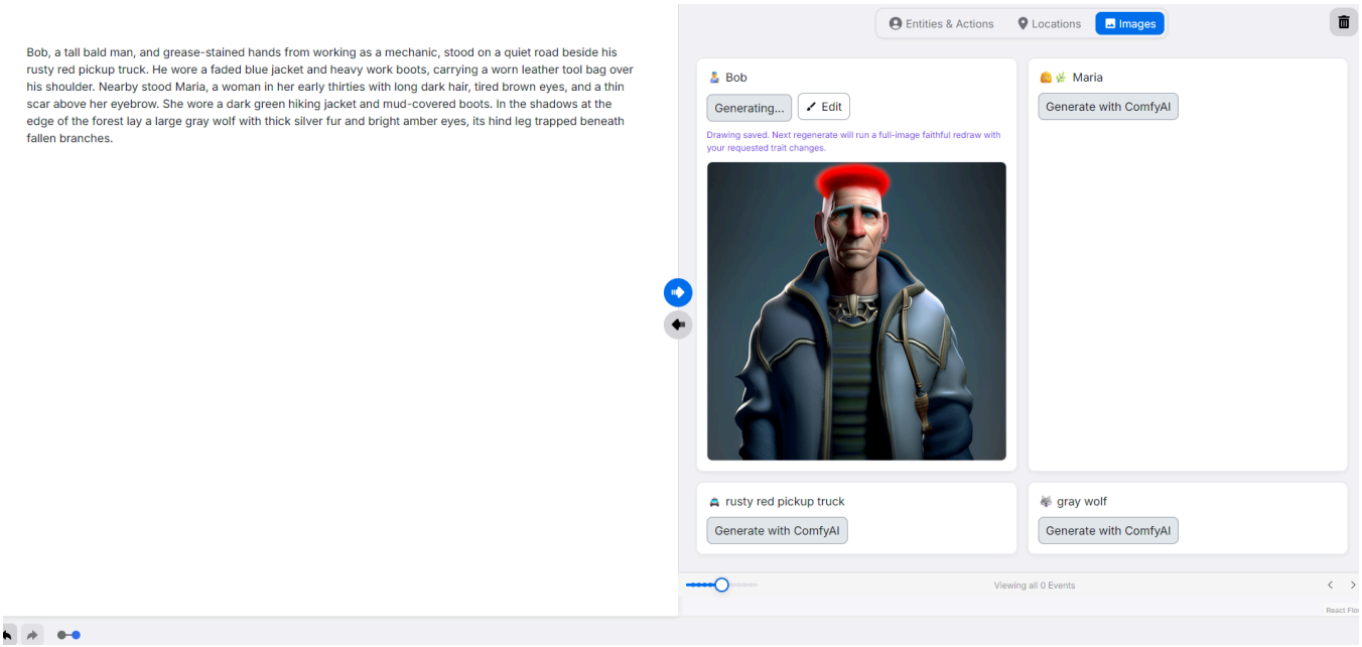
Tâche 7: Adapter et contourner le problème de regénération pour ComfyAI

Le besoin a évolué au cours du développement. Initialement, l'objectif était de modifier uniquement une zone précise de l'image à l'aide d'un masque, et techniquement, ce système a fini par fonctionner correctement. Cependant, les résultats obtenus n'étaient pas suffisamment fiables ni cohérents avec le besoin métier. Les modifications restaient trop locales, souvent imprécises, et ne permettaient pas d'obtenir un rendu global crédible.

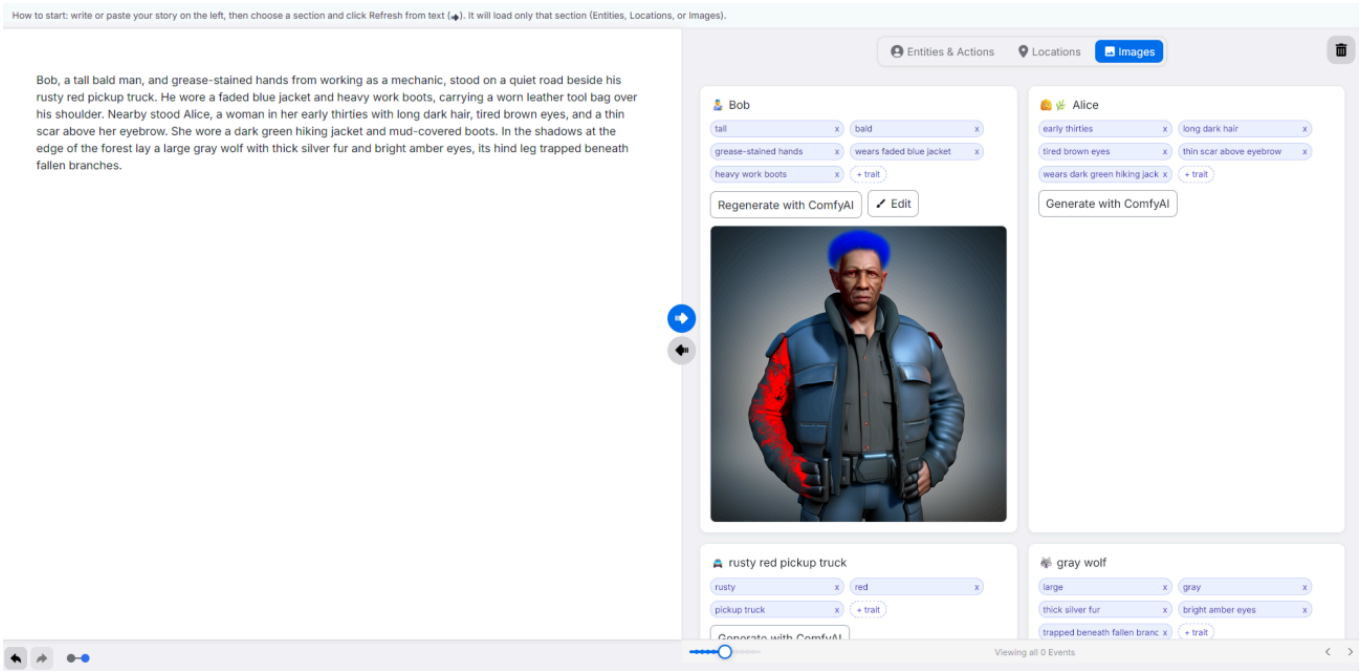
J'ai donc pris la décision de changer d'approche et de passer à une régénération complète de l'image. J'ai introduit un mode que j'ai appelé GLOBAL REDRAW MODE, dans lequel le masque n'est plus utilisé comme contrainte principale, mais plutôt comme un indicateur d'intention via le dessin utilisateur. Le dessin devient alors un guide visuel qui informe l'IA sur la forme, la couleur et l'objectif de modification, tandis que l'image est entièrement régénérée.

Dans ce mode, l'IA reconstruit l'image en conservant les éléments essentiels comme l'identité du personnage, le style visuel, la pose et la composition, tout en intégrant les nouvelles modifications demandées. Cette approche permet d'éviter les incohérences locales causées par l'édition par masque et produit des résultats beaucoup plus naturels.

Afin de stabiliser le rendu, j'ai également renforcé le prompt système pour que ce soit aussi complet que nécessaire pour que l'IA comprenne mes besoins.



Le résultat correspond désormais au besoin. Par exemple, un gribouillis rouge sur la tête accompagné du prompt “cheveux rouges bouclés” produit des cheveux satisfaisants et réalistes, sans traces du dessin initial, contrairement à l’approche par masque qui donnait souvent des résultats partiels ou incorrects. Cela étant dit, arriver à un résultat satisfaisant pour ça m'a coûté la consistence du reste du dessin, car des fois l'IA hallucine et rajoute des détails qui n'étaient pas présents, ou change des traits majeurs, bien que cela s'avère être positif, car exemple mon personnage passait d'être blanc à noir, mais comme dans le texte aucune description n'indiquait sa couleur de peau, ça incite l'auteur à se rendre compte qu'il n'a pas donné suffisamment de détails physiques sur l'entitée.



Tâche 8: Réactiver les labels de traits qui fonctionnait bien et reconnexion avec l'éditeur de texte.

Une fois cette nouvelle approche stabilisée, j’ai réintroduit le système de traits (labels), qui permet de modifier les caractéristiques des entités de manière efficace.

J’ai donc réintégré dans `ImagesEditor.tsx` l’affichage des traits, leur modification inline, ainsi que les fonctionnalités d’ajout et de suppression. La génération d’image a été adaptée pour utiliser ces traits lors de la construction du prompt, en combinant le nom de l’entité, ses attributs et un style global. Le système de labels fonctionne désormais en parallèle du dessin, chacun pouvant influencer la génération finale.

J’ai également ajouté une synchronisation automatique entre les modifications effectuées via dessin et les labels. Lorsqu'un éditage dessin modifie un trait du personnage, les labels sont ajustés parallélement pour rester cohérent.

J’ai aussi ajouté une télémétrie (`durationMs`) dans les réponses backend afin de mieux suivre les performances.

Enfin, j’ai reconnecté ce nouveau système avec celui déjà existant dans la codebase, qui permet à l’IA de proposer des modifications du texte soumis en fonction des changements effectués sur les entités.